

UNITED STATES PROVISIONAL PATENT APPLICATION

Provisional Application for Patent - 35 U.S.C. ss 111(b)

Applicant: Suvojeet Pal (Individual)

Inventor: Suvojeet Pal

Docket Reference: AVERIO-001-US-PROV

Application Type: Provisional Application for Patent

Filed Under: 35 U.S.C. § 111(b)

Applicant: Suvojeet Pal (Individual)

Inventor: Suvojeet Pal

Correspondence Address: [Inventor Address]

Docket Reference: AVERIO-001-US-PROV

TITLE OF THE INVENTION

ADAPTIVE MULTI-STRATEGY ENTITY RESOLUTION SYSTEM WITH PROBABILISTIC CLUSTER DRIFT DETECTION, PROVISIONAL GOLDEN IDENTITY MANAGEMENT, AND SELF-CALIBRATING FELLEGI-SUNTER PARAMETER ESTIMATION FOR ENTERPRISE MASTER DATA MANAGEMENT

FIELD OF THE INVENTION

This invention is in the field of enterprise software — specifically, systems that manage and deduplicate customer and business entity records across multiple source systems at large scale. The core problem the invention solves is: given hundreds of millions of records arriving from different systems over time, how do you reliably determine which records refer to the same real-world person or organization, keep that determination accurate as data changes, and guarantee that every record is always identifiable — even before a human has reviewed it?

BACKGROUND

How I Came to Build This

I built Averio MDM after years of working in data management and watching enterprises struggle with the same fundamental problem: their customer data lived in a dozen different systems, and no two systems agreed on who was who. A customer named "Robert Johnson" in the CRM was "Bob Johnson" in the billing system and "R. Johnson" at the same address in the loan platform. Identity resolution — figuring out that these three records are the same person — was either done by expensive consulting projects with static rule sets, or not done at all.

The commercial MDM products that existed (IBM InfoSphere, Informatica, SAP MDG, Reltio) all shared the same architectural limitations that I kept running into:

They block candidates using too few strategies. Most systems pick one or two blocking strategies — typically phonetic name and postal code. If a name is spelled differently in two systems, or a customer moved, those records never even get compared. The miss rate on real enterprise data is surprisingly high.

Their match models are frozen at deployment. The statistical parameters that determine how much weight to give a first-name match versus a phone match are set once by a consultant and never updated. As data quality improves over time, as new source systems come in, as the demographics of the customer base shift — the model just gets stale. Nobody re-runs the calibration.

They leave records in a limbo state. When a record can't be auto-matched and gets sent to a human reviewer, it sits in a queue with no identity. It can't be found by golden ID, it doesn't appear in reports, downstream applications that query by customer master ID get nothing back. This can last days or weeks. For a bank, that means a customer who just opened an account can't be seen by the risk system yet.

They don't notice when a cluster goes wrong. Once two records get merged into a cluster, they stay merged even if the attributes that caused the merge are later corrected. The phone number that linked two records together gets fixed — but the incorrect merge stays. Nobody detects this unless a human happens to look.

They operate in a single matching mode. Either you write rules, or you configure a statistical model. There's no system that falls through to an AI-based disambiguation for the cases that sit right on the fence.

These are real, specific problems I encountered repeatedly. Averio MDM is my attempt to fix all of them in one coherent system.

Why Existing Approaches Fall Short

On blocking: A naive pairwise comparison of N records is $O(N^2)$. For a billion-record enterprise that's 10^{18} comparisons — not workable. Blocking narrows the search, but single-strategy blocking misses too many real matches when data quality varies across systems. Two records for the same person won't share a blocking key if one system truncated the last name and the other stored an old address. The solution I developed uses nine independent strategies simultaneously, taking their union so that any single strategy match is sufficient to bring a candidate into scope.

On self-calibrating parameters: The Fellegi-Sunter model assigns weights based on two probabilities per attribute: the probability of agreement given a true match (m-probability) and the probability of agreement given a true non-match (u-probability). In prior systems, someone estimates these once. I implemented an Expectation-Maximization algorithm that re-learns these probabilities from the actual data on a nightly schedule, with no labeled training pairs required. The model stays accurate as data characteristics change.

On the null golden state: Every existing commercial MDM system I reviewed has this gap. A record in review has no golden ID. I fixed it with what I call the Provisional Golden Identity pattern — every record gets a golden ID at the moment it's first written to the database, period. If a human later approves a merge, the provisional ID is retired. If they reject, the provisional becomes permanent. Either way the record is always findable.

On cluster drift: Nobody has published a solution to post-assignment drift in MDM systems. When I looked through prior art, I found papers on the initial matching problem and on survivorship, but nothing on what happens when the attributes that justified a cluster membership change. I built a re-evaluation algorithm that fires on every attribute update and checks whether the updated record still statistically belongs to its cluster.

SUMMARY OF THE INVENTION

The invention is a complete enterprise MDM platform called Averio MDM, implemented as a Java Spring Boot application. It introduces five technical innovations that I believe are novel to the field:

1. Nine-Strategy Union Blocking. Instead of one or two blocking strategies, the system generates keys from nine independent strategies and takes their union. Any single strategy overlap is enough to bring a candidate into scope. This dramatically improves recall on messy real-world data without sacrificing the $O(N \times k)$ complexity bound.

2. Self-Calibrating EM Parameter Estimation. The Fellegi-Sunter match model calibrates itself nightly using an Expectation-Maximization algorithm. It learns from the

actual data in the system — no labeled training pairs, no consultant re-engagement. The model adapts as data quality improves and as new source systems are added.

3. Three-Stage Cascading Match Pipeline. Records flow through three matching stages: deterministic (exact identifier match), probabilistic (Fellegi-Sunter weighted scoring), and AI-enhanced (Large Language Model disambiguation for ambiguous cases only). The LLM is invoked only for scores in the uncertain range [0.5, 0.9], preserving the speed and accuracy of the first two stages for the majority of decisions.

4. Provisional Golden Identity. Every record gets a globally unique golden identifier before any database transaction commits. There is no null golden state at any point in the record lifecycle. Records pending review are fully available to downstream applications, the timeline service, the survivorship engine, and the blocking index from the moment they are written.

5. Post-Update Cluster Drift Detection. After every attribute update, the system re-evaluates whether the updated record still statistically belongs to its current cluster. If it has drifted, the system automatically reassigns it, escalates to human review, or creates a new cluster — and records the entire event chain as immutable timeline documents.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 is a system architecture diagram showing the Averio MDM platform components — the blocking engine, three-stage match pipeline, survivorship engine, and data stores (Neo4j graph database and Azure Cosmos DB).

FIG. 2 is a flowchart showing how the nine blocking key strategies are applied to a single party record to produce the candidate pool.

FIG. 3 is a flowchart of the three-stage cascading match pipeline showing threshold-driven routing to auto-link, steward review, and new entity creation.

FIG. 4 is a diagram of the Expectation-Maximization parameter estimation algorithm — the E-step computing posterior match probabilities and the M-step updating m and u estimates.

FIG. 5 is a flowchart of the post-update cluster drift detection and remediation process.

FIG. 6 is a data model diagram showing the relationship between Party nodes in Neo4j, GoldenRecord documents, and TimelineEvent documents in Cosmos DB.

FIG. 7 is a flowchart of the provisional golden identity lifecycle — from first ingest through steward merge approval or rejection.

FIG. 8 is a diagram of the human-in-the-loop ML retraining feedback cycle showing feature capture at decision time and automatic logistic regression retraining.

DRAWINGS

(Referenced in Brief Description of the Drawings)

FIG. 1

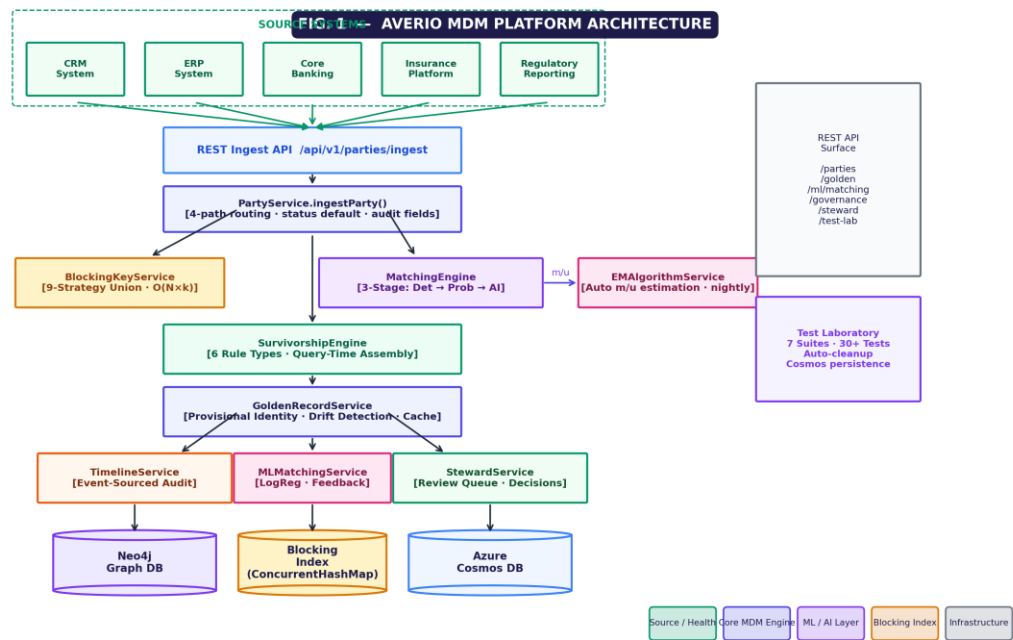


FIG. 1 — AVERIO MDM PLATFORM — Full System Architecture

FIG. 1 — System Architecture: Overall AVERIO MDM platform showing the blocking engine, three-stage match pipeline, survivorship engine, and dual data stores (Neo4j + Azure Cosmos DB).

FIG. 2

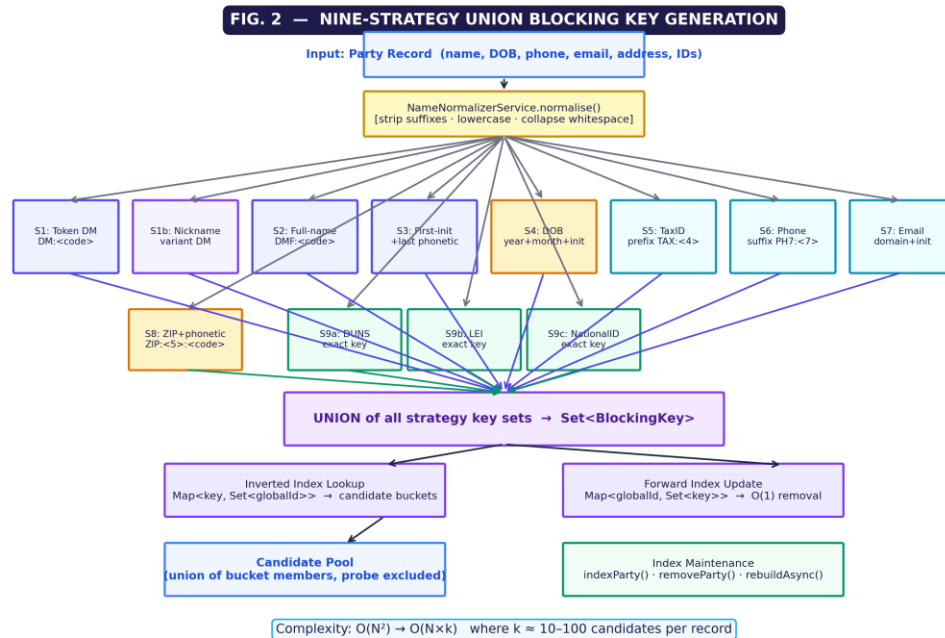


FIG. 2 — Nine-Strategy Union Blocking Key Generation (BlockingKeyService)

FIG. 2 — Nine-Strategy Union Blocking: Flowchart of the nine independent blocking key generation strategies applied in union for $O(N \times k)$ candidate retrieval.

FIG. 3

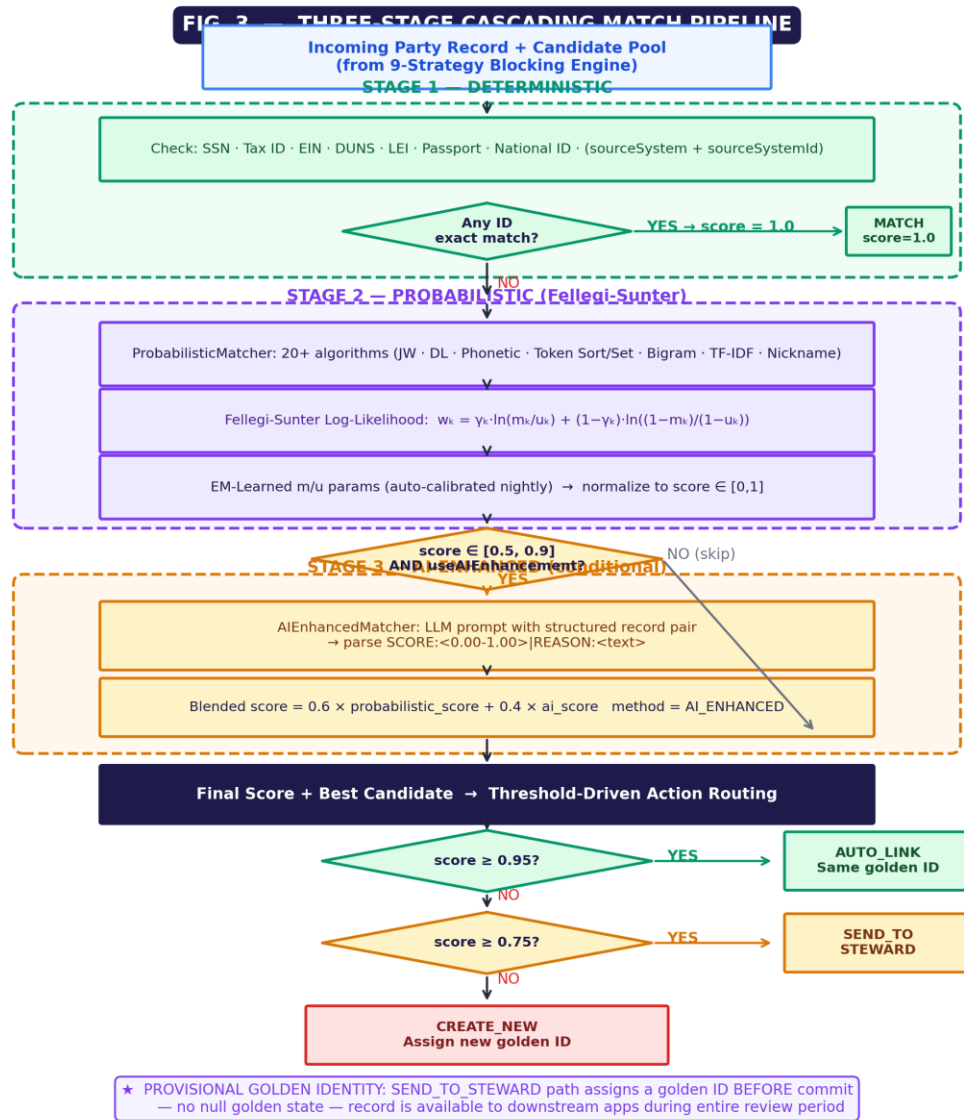


FIG. 3 — Three-Stage Cascading Match Pipeline (MatchingEngine)

FIG. 3 — Three-Stage Cascading Match Pipeline: Threshold-driven routing through deterministic, probabilistic Fellegi-Sunter, and AI-enhanced LLM matching stages.

FIG. 4

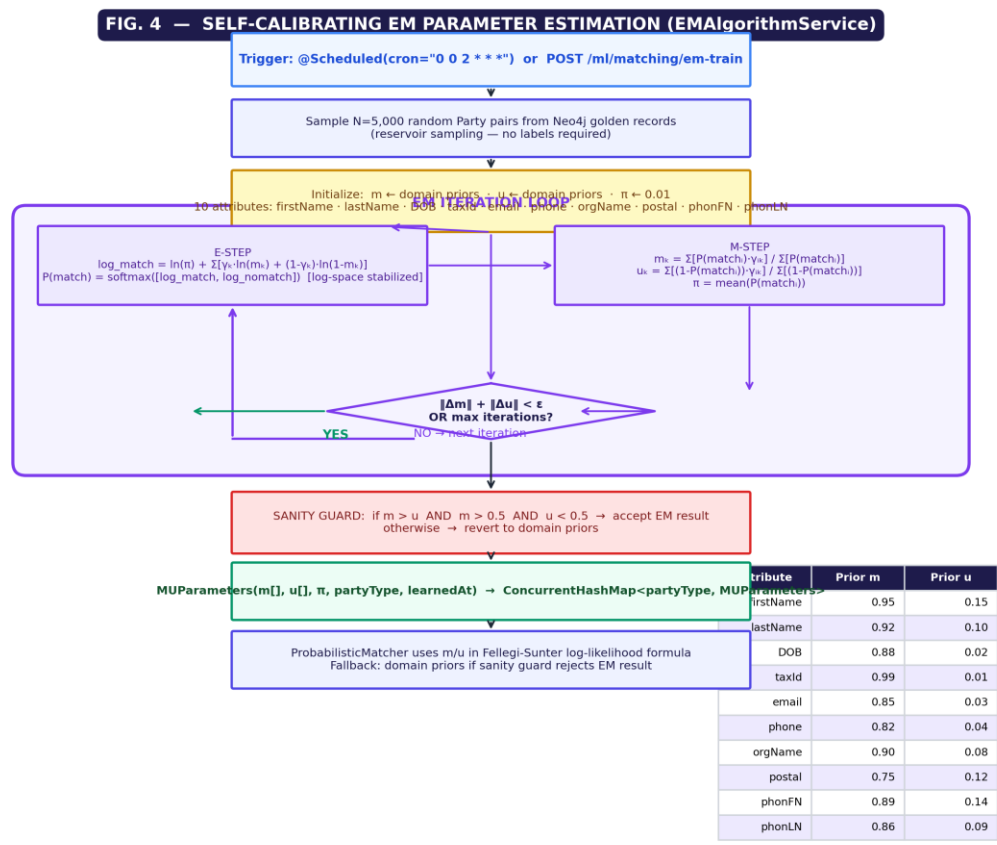


FIG. 4 — EM Parameter Estimation for Fellegi-Sunter m/u Probabilities

FIG. 4 — Self-Calibrating Fellegi-Sunter EM Algorithm: E-step and M-step iterations for autonomous m-probability and u-probability estimation across ten attributes.

FIG. 5

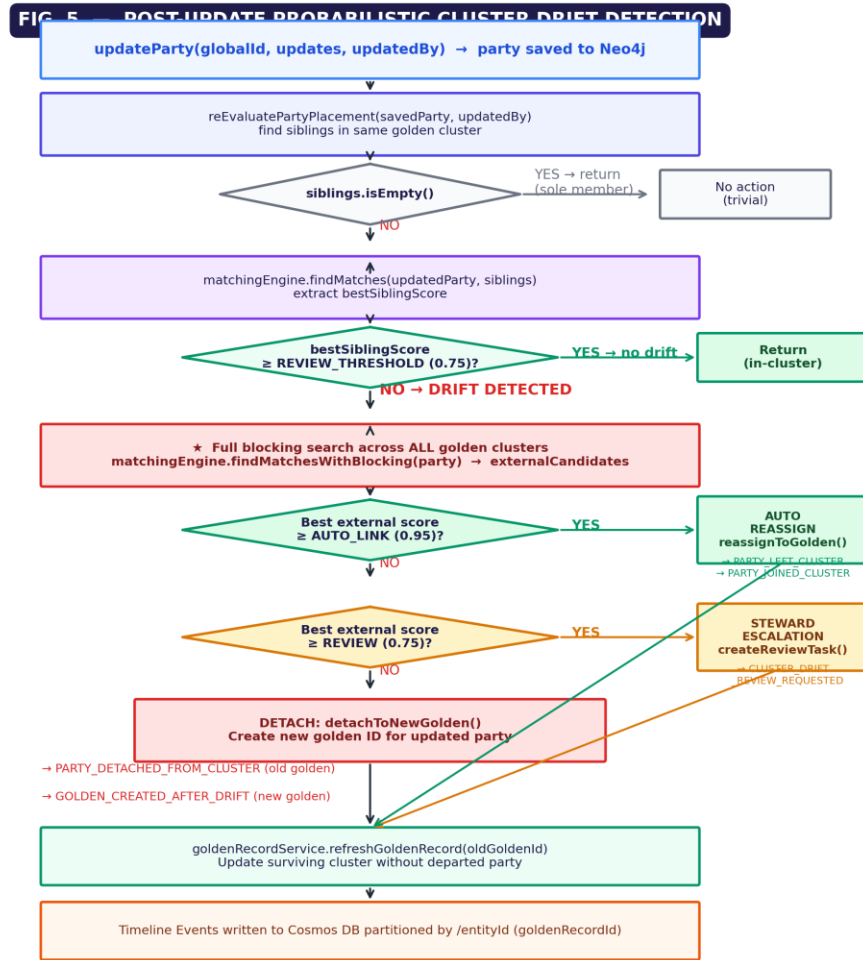


FIG. 5 — Post-Update Probabilistic Cluster Drift Detection (PartyService.reEvaluatePartyPlacement)

FIG. 5 — Post-Update Cluster Drift Detection: Probabilistic re-evaluation of cluster membership following attribute update, with automated reassignment and steward escalation.

FIG. 6

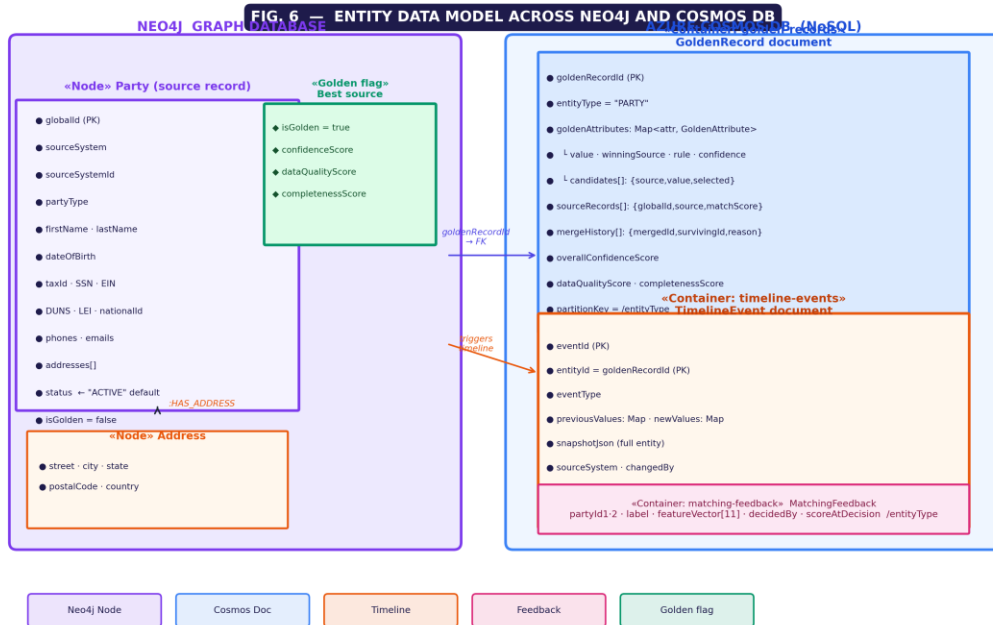


FIG. 6 — Entity Data Model — Neo4j (graph) and Azure Cosmos DB (documents)

FIG. 6 — Hybrid Graph + Document Data Model: Relationships between Party nodes (Neo4j), GoldenRecord documents, and TimelineEvent documents (Azure Cosmos DB).

FIG. 7

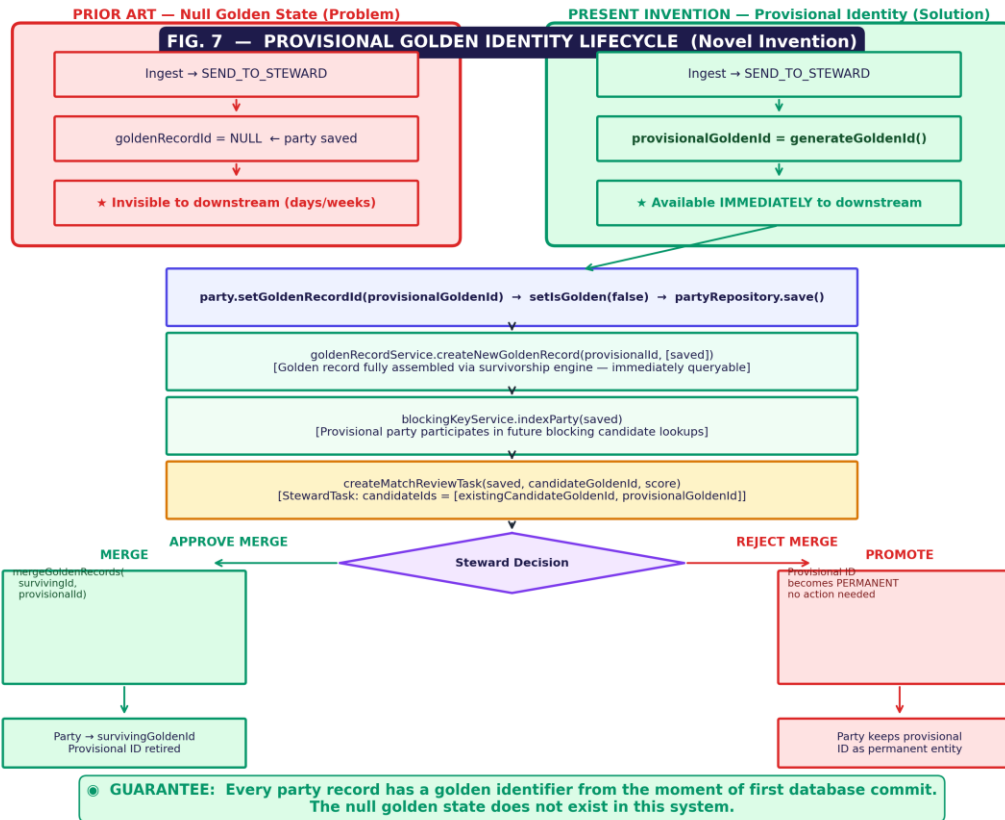


FIG. 7 — Provisional Golden Identity Lifecycle — eliminates null golden state (Novel)

FIG. 7 — Provisional Golden Identity Lifecycle: Assignment of a globally unique golden identifier at first persistence, eliminating the null golden state throughout steward review.

FIG. 8

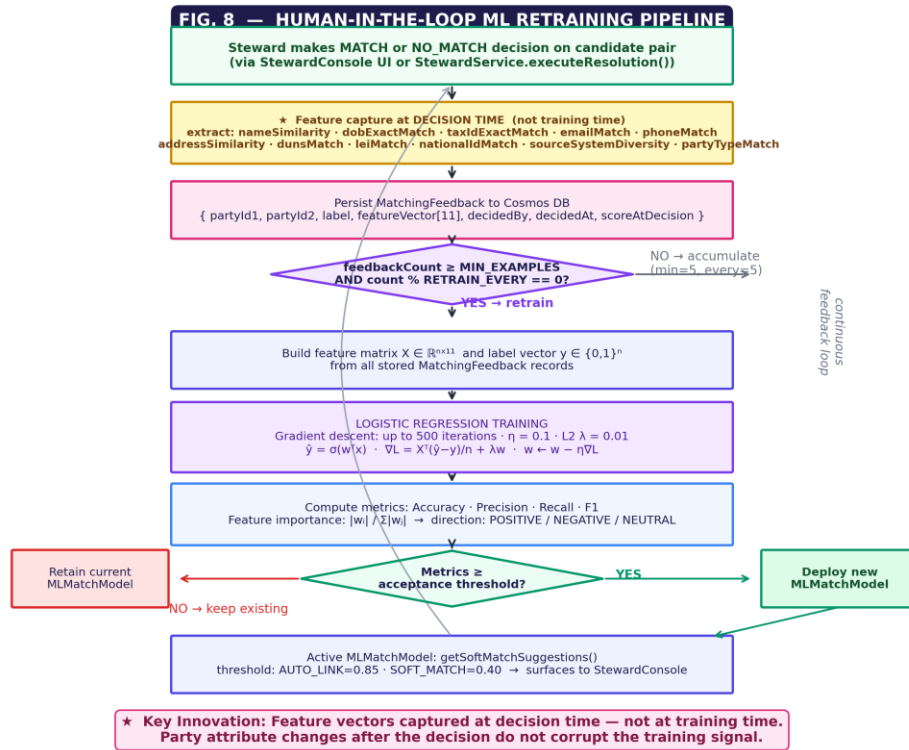


FIG. 8 — Human-in-the-Loop ML Retraining Pipeline (MLMatchingService)

FIG. 8 — Human-in-the-Loop ML Feedback Pipeline: Feature vector capture at steward decision time, immutable feedback storage, and automatic logistic regression retraining.

DETAILED DESCRIPTION

I. SYSTEM ARCHITECTURE

Averio MDM runs as a Spring Boot 3.x application on Java 21, deployed on Microsoft Azure. The data layer uses two stores deliberately chosen for different access patterns:

- Neo4j (graph database) stores Party nodes and their relationships. Graph traversal is used for hierarchy resolution, relationship management, and blocking index candidate retrieval when party relationships are relevant.
- Azure Cosmos DB (document database) stores golden records, timeline events, matching feedback, survivorship rules, steward tasks, and audit logs. Document storage is appropriate here because these are semi-structured, append-heavy, and queried by ID.
- In-memory concurrent hash maps store the blocking index. The index must support microsecond lookup times that a database cannot provide at scale.

The platform's principal services are:

- `PartyService` — ingest pipeline, update handling, drift detection trigger
- `BlockingKeyService` — nine-strategy key generation, dual-map index maintenance
- `MatchingEngine` — orchestrates the three-stage pipeline
- `ProbabilisticMatcher` — Fellegi-Sunter weighted scoring with 20+ similarity algorithms
- `AIEnhancedMatcher` — LLM-based disambiguation stage
- `EMAlgorithmService` — nightly self-calibrating parameter estimation
- `SurvivorshipEngine` — query-time golden record assembly from source records
- `GoldenRecordService` — golden record creation, refresh, and view-scoped retrieval
- `MLMatchingService` — feature capture, logistic regression training, and model scoring
- `StewardService` — human review task lifecycle management
- `TimelineService` — immutable event recording for all state changes
- `ClusterMergeService` — transitive Union-Find cluster formation

II. NINE-STRATEGY UNION BLOCKING ENGINE

II.A. Why Nine Strategies

Single-strategy blocking fails when data quality varies. Consider a customer record where:

- The name is spelled "Johnsen" in the CRM but "Johnson" in the billing system
- The phone number stored in each system has a different area code

- The address was updated in one system but not the other

With phonetic name blocking alone, these two records share no bucket if the phonetic encoding diverges. With address blocking alone, they miss. But with nine strategies, the shared email domain, the shared last-seven digits of a backup phone number, or the shared tax ID prefix will create at least one matching bucket — and the union of all strategy buckets is the candidate pool.

The performance cost of nine strategies is minimal because keys are short strings and hash map lookups are $O(1)$. The recall improvement on real enterprise data is substantial.

II.B. The Nine Strategies

For each incoming `Party` record, `BlockingKeyService.generateKeys(Party party)` produces a set of string keys:

Strategy 1 — Double Metaphone of Name Tokens (`DM:<code>`):

Each whitespace-separated token in the party's name (using `organizationName` for organizations, `firstName + lastName` for individuals) is encoded with the Double Metaphone algorithm. Each code is prefixed `DM:` and added to the key set. "Johnson" and "Jonsen" both produce `DM:JNSN`.

Strategy 1b — Nickname Variant Phonetic Keys:

For individual records, `NicknameService.variants(firstName)` returns all equivalent name forms from a table of 90+ equivalence groups. Each variant gets its own `DM:` key. This means "Bob Smith" and "Robert Smith" share at least one blocking key even though "Bob" and "Robert" produce different phonetic codes.

Strategy 2 — Collapsed Full Name Phonetic (`DMF:<code>`):

All whitespace is stripped from the normalized name before Double Metaphone encoding. "John Smith" becomes "JohnSmith" before encoding. This handles cases where token boundaries differ between source systems.

Strategy 3 — First Initial Plus Last Token Phonetic (`FI:<c>:<code>`):

The first character of the first name and the Double Metaphone code of the last name token are combined. "J:SM0" would be a typical key. Nickname expansion is applied to the first token, generating additional FI: keys per variant.

Strategy 4 — Date-of-Birth Year-Month Plus Name Initial (`DOB:<yyyy>-<mm>:<c>`):

For parties with a known date of birth, year, month, and the initial of the primary name are combined. This is highly discriminating — it only matches parties born in the same month with the same name initial.

Strategy 5 — Tax Identifier Prefix (`TAX:<4digits>`):

The first four digits of the Tax ID or EIN (after stripping hyphens and spaces) form a blocking key. This is effective for organizations where the full tax ID is not always populated.

Strategy 6 — Phone Number Suffix (`PH7:<7digits>`):

The last seven digits of each phone number across all of the party's phone entries are used as blocking keys. Seven-digit suffixes are consistent across area codes, which often differ between source systems.

Strategy 7 — Email Domain Plus Name Initial (`EM:<domain>:<c>`):

The domain portion of the email address (the part after `@`) and the first character of the primary name are combined. This works well for corporate records where multiple contacts share a company domain.

Strategy 8 — Postal Code Prefix Plus Phonetic (`ZIP:<5chars>:<code>`):

The first five characters of the primary address postal code and the Double Metaphone code of the first name token are combined. This scopes phonetic matches geographically.

Strategy 9 — Exact High-Cardinality Identifiers (`DUNS:<#>`, `LEI:<#>`, `NID:<#>`):

DUNS number, Legal Entity Identifier, and National ID are stored verbatim as blocking keys after character normalization. A bucket hit on these is essentially a definite match signal on its own.

II.C. Dual-Map Index

Two `ConcurrentHashMap` instances are maintained in application memory:

```
inverted_index: Map<String(blockingKey), Set<String(globalId)>>
forward_index: Map<String(globalId), Set<String(blockingKey)>>
```

The forward index exists specifically to support efficient deletion. When a party is deleted or merged, `removeParty(globalId)` uses the forward index to find all of that party's keys in $O(|\text{keys}|)$ time, then removes them from the inverted index. Without the forward index, deletion would require a full scan of the inverted index.

The candidate pool for any incoming record is the union of all bucket members across all strategy keys, minus the probe record itself.

III. SELF-CALIBRATING FELLEGI-SUNTER PARAMETER ESTIMATION

III.A. The Core Model

The Fellegi-Sunter framework assigns a composite weight to a comparison between two records. For each attribute i , the log-likelihood ratio contribution is:

$$w_i = \gamma_i \times \ln(m_i / u_i) + (1 - \gamma_i) \times \ln((1 - m_i) / (1 - u_i))$$

where γ_i is 1 if the attribute agrees and 0 if it disagrees. The total weight is the sum across all attributes. A high total weight means the pair is likely a true match; a low weight means it is likely not.

The critical inputs are m_i (probability of agreement given the pair is a true match) and u_i (probability of agreement given the pair is not a true match). In every prior MDM system I reviewed, these are set manually and never updated. I built the EM algorithm to learn them automatically.

III.B. What the EM Algorithm Does

`EMAlgorithmService` runs nightly at 02:00 UTC. For each party type in the system, it:

1. Draws a sample of 5,000 random party pairs from Neo4j using reservoir sampling.
2. Computes a 10-dimensional binary agreement vector for each pair across these attributes:
 - firstName (index 0): Jaro-Winkler ≥ 0.85
 - lastName (index 1): Jaro-Winkler ≥ 0.85
 - dateOfBirth (index 2): exact match
 - taxId (index 3): exact match after normalization
 - email (index 4): exact match after lowercasing
 - phone (index 5): last-7-digit match
 - organizationName (index 6): Jaro-Winkler ≥ 0.80
 - postalCode (index 7): first-5-character match
 - phonetic firstName (index 8): Double Metaphone match
 - phonetic lastName (index 9): Double Metaphone match
3. Initializes m and u to domain expert priors:

```
m (P(agree | match)):      [0.95, 0.92, 0.88, 0.99, 0.85, 0.82, 0.90, 0.75,
0.89, 0.86]
u (P(agree | non-match)): [0.15, 0.10, 0.02, 0.01, 0.03, 0.04, 0.08, 0.12,
0.14, 0.09]
```

4. E-step: For each pair, compute the posterior probability it is a true match using numerically stable log-space arithmetic:

```
log_match   = ln(n) + Σ_k [γ_k × ln(m_k) + (1-γ_k) × ln(1-m_k)]
log_nomatch = ln(1-n) + Σ_k [γ_k × ln(u_k) + (1-γ_k) × ln(1-u_k)]
maxLog      = max(log_match, log_nomatch)
P(match)    = exp(log_match - maxLog) / [exp(log_match - maxLog) +
exp(log_nomatch - maxLog)]
```

The `maxLog` subtraction prevents floating-point underflow in log-space, which I found was a real problem with naive implementations on large attribute vectors.

5. M-step: Update the parameters from the posterior-weighted observations:

$$\begin{aligned} m_k &= \sum_i [P(\text{match}_i) \times y_{ik}] / \sum_i [P(\text{match}_i)] \\ u_k &= \sum_i [(1 - P(\text{match}_i)) \times y_{ik}] / \sum_i [(1 - P(\text{match}_i))] \\ \pi &= \text{mean}(P(\text{match}_i)) \end{aligned}$$

6. Iterate until convergence ($\|m_{\text{new}} - m_{\text{old}}\|_1 + \|u_{\text{new}} - u_{\text{old}}\|_1 < 0.001$) or 100 iterations maximum.
7. Validate before accepting: if ``m_k > u_k && m_k > 0.5 && u_k < 0.5`` for all k, accept. Otherwise revert to priors. This guard prevents statistically aberrant results from a sparse early data sample from degrading match quality.
8. Store results per party type with timestamp in a ``ConcurrentHashMap<String, MUParameters>``.

IV. THREE-STAGE CASCADING MATCH PIPELINE

IV.A. Stage 1 — Deterministic Matching

The first stage is not probabilistic — it's a lookup. If the probe record and a candidate share any of: SSN, Tax ID/EIN, DUNS number, LEI, passport number, or national ID (exact match after normalization), the score is set to 1.0 and the match method is recorded as `DETERMINISTIC`. There is also a deduplication check: if the probe and candidate share both `sourceSystem` and `sourceSystemId`, the score is 1.0 and method is `SOURCE\_DEDUP`. This prevents re-ingesting the same source record twice.

Stage 1 exits immediately for any deterministic match — it doesn't proceed to Stage 2.

IV.B. Stage 2 — Probabilistic Fellegi-Sunter Scoring

`ProbabilisticMatcher` applies 20+ similarity algorithms across attribute categories, maps each comparison to a Fellegi-Sunter log-likelihood contribution using the EM-learned m and u parameters, and normalizes the total to [0, 1].

The similarity algorithms include:

- Jaro-Winkler for name fields (first name, last name, organization name)
- Token Sort Ratio: alphabetically sort tokens before Jaro-Winkler — handles word-order differences ("Smith, John" vs "John Smith")
- Token Set Ratio: Jaccard similarity over token sets plus Jaro-Winkler — handles abbreviations
- Bigram Jaccard: 2-character overlap ratio
- Trigram Jaccard: 3-character overlap ratio

- Composite name similarity: maximum of multiple metrics with a phonetic boost when Double Metaphone codes match
- Damerau-Levenshtein / Optimal String Alignment: handles the four edit operations (substitution, insertion, deletion, transposition) for catching typos
- TF-IDF cosine similarity over character n-gram vectors
- Double Metaphone phonetic match
- Nickname equivalence: 1.0 if names are known equivalents (e.g., Bob/Robert), 0.0 otherwise

IV.C. Stage 3 — AI-Enhanced Disambiguation

Stage 3 is only invoked when: (a) the probabilistic score is in the ambiguous range [0.5, 0.9] and (b) the active matching rule has `useAIEnhancement = true`.

`AIEnhancedMatcher.score(partyA, partyB)` constructs a structured prompt with both records' key attributes and requests a numeric score and brief rationale:

```
SCORE:<0.00-1.00>|REASON:<brief explanation>
```

The final blended score is $0.6 \times \text{probabilistic_score} + 0.4 \times \text{ai_score}$. The LLM adds value specifically in cases that are ambiguous to pure statistical methods — for example, distinguishing "IBM" from "IBM Global Services" when the tax IDs are not populated and the address is slightly different.

Using LLM matching for the full candidate pool would be too expensive and too slow. Using it only for the uncertain range makes the cost bounded and predictable.

IV.D. Routing Thresholds

After scoring, the top candidate's score drives the routing decision:

| Score | Action |

|---|---|

| ≥ 0.95 | AUTO_LINK — merge to existing cluster |

| $0.75 - 0.94$ | SEND_TO_STEWARD — create steward review task |

| < 0.75 | CREATE_NEW — independent new entity |

Thresholds are configurable per `MatchingRule` entity, allowing different sensitivity for different source systems or entity types.

V. PROVISIONAL GOLDEN IDENTITY MANAGEMENT

V.A. The Problem in Concrete Terms

I want to be specific about what "null golden state" means in practice, because I think it's an underappreciated problem.

When a record is sent to steward review in a conventional MDM system:

- The risk system queries by golden customer ID — gets nothing back
- The compliance system can't record an activity against this customer
- If another record comes in for the same person in the next 10 minutes, it goes to a second review task — but neither task knows about the other
- Timeline events can't be indexed by golden ID because there is none
- Downstream data warehouse feeds have a gap

This is not a hypothetical edge case. In a large bank processing 50,000 new customer records per day, there may be thousands of records in steward review at any given time. The null golden state affects all of them.

V.B. The Solution

When `PartyService.ingestParty()` routes a new record to `SEND_TO_STEWARD`:

9. `generateGoldenId()` is called — produces a zero-padded random numeric string (e.g., `'0000000012847593'`)
10. The incoming party is assigned this provisional golden ID:
`'incoming.setGoldenRecordId(provisionalId)'`
11. The party is saved to Neo4j with `'isGolden = false'` (marking it as a source record, not the cluster representative)
12. `'GoldenRecordService.createNewGoldenRecord(provisionalId, List.of(savedParty))'` is called immediately — a complete golden record is assembled with survivorship rules applied from this single source record
13. The provisional record is added to the blocking index: `'indexParty(savedParty)'`
14. A steward task is created with `'candidateIds = [existingCandidateGoldenId, provisionalGoldenId]'`

From this point, `'provisionalId'` works exactly like any permanent golden ID. The record is retrievable, it can receive timeline events, it shows up as a candidate for future matches, and the survivorship engine can assemble a golden view from it.

If the steward approves the merge:

`'StewardService.executeResolution()'` calls `'mergeGoldenRecords(survivingId, provisionalId)'`. The party record is re-pointed to `'survivingId'`. The provisional golden record is deleted or archived. The surviving cluster's golden record is refreshed.

If the steward rejects the merge:

No action needed. `'provisionalId'` becomes the permanent golden ID for this record. The review task is closed.

The guarantee: Every party record written to the database has a valid golden identifier at all times. There is no window — not even for a fraction of a second — where a record exists without an identity.

V.C. Client-Supplied Identity Migration

For migrating from legacy systems with existing identifiers:

```
POST /api/v1/parties/ingest
{ "goldenRecordId": "LEGACY-CUST-10004231", ... }
```

When `goldenRecordId` is non-blank in the ingest payload, the matching engine is skipped entirely and the supplied identifier is used as the golden ID. This allows organizations to migrate existing customer master data into Averio MDM without losing their existing identity keys.

VI. POST-UPDATE CLUSTER DRIFT DETECTION

VI.A. The Problem

Here's a scenario that happens more often than it should:

Two records for different entities both had the phone number `+1-212-555-0100` in the source systems. They were merged into the same cluster on the strength of that phone match. Later, one record gets corrected — it turns out that was a data entry error, the real number is `+1-415-555-0200`. The cluster membership is now wrong. The records no longer have matching attributes, but the merge stays.

In every commercial MDM system I looked at, this stays wrong indefinitely unless a data steward happens to manually review it. There's no automated detection of this condition.

I built `PartyService.reEvaluatePartyPlacement()` specifically to catch this.

VI.B. The Detection Algorithm

`reEvaluatePartyPlacement(Party party, String updatedBy)` is called at the end of every `updateParty()` operation.

Step 1 — Score against current cluster siblings:

```
List<Party> siblings = partyRepository.findByGoldenRecordId(currentGoldenId)
    .stream()
    .filter(s -> !s.getGlobalId().equals(party.getGlobalId()))
    .collect(Collectors.toList());
```

Run the matching engine against all siblings. Record `bestSiblingScore`.

Step 2 — Threshold check:

If ``bestSiblingScore >= 0.75``, the updated record still statistically belongs in its cluster. Return. Nothing to do — this is the common case and it exits cheaply.

Step 3 — External candidate search (drift confirmed):

If ``bestSiblingScore < 0.75``, drift is confirmed. Run full blocking + scoring against all golden clusters:

```
MatchingEngine.MatchResult externalResult =  
matchingEngine.findMatchesWithBlocking(party, null);
```

Filter to candidates in a **different** golden cluster than the current one.

Step 4 — Remediation:

| Condition | Action |

|---|---|

| External score ≥ 0.95 | Auto-reassign to external cluster |

| External score 0.75–0.94 | Send to steward for review |

| No qualifying external match | Detach to new golden cluster |

Step 5 — Refresh old cluster:

After any branch, ``goldenRecordService.refreshGoldenRecord(currentGoldenId)`` recomputes the surviving cluster's golden record without the departed party.

VI.C. Timeline Events for Cluster Changes

Every cluster membership change is recorded as an immutable event document in Cosmos DB, partitioned by golden record ID. This creates a complete audit trail of cluster history:

- ``PARTY_LEFT_CLUSTER`` — recorded on the old golden ID
- ``PARTY_JOINED_CLUSTER`` — recorded on the new golden ID
- ``CLUSTER_DRIFT_REVIEW_REQUESTED`` — recorded on the old golden ID when sent to steward
- ``PARTY_DETACHED_FROM_CLUSTER`` — recorded on the old golden ID
- ``GOLDEN_CREATED_AFTER_DRIFT`` — recorded on the new golden ID

These events enable point-in-time reconstruction of which records belonged to which cluster at any historical date — something compliance and audit teams in financial services need regularly.

VII. TRANSITIVE CLUSTER MERGING

`ClusterMergeService` handles the transitive closure problem: if A matches B and B matches C, all three belong in the same cluster even if A and C were never directly compared.

The Union-Find (Disjoint Set Union) algorithm handles this:

15. Initialize each record as its own root.
16. For each pair (A, B) with score above the link threshold, call `union(A, B)`.
17. Path compression and union-by-rank keep each `find()` at effectively $O(1)$ amortized (the inverse Ackermann function $\alpha(N) \approx 4$ for any realistic N).
18. `getClusters()` groups records by their root representative.

This guarantees no two records that are transitively connected end up in separate golden clusters.

VIII. QUERY-TIME SURVIVORSHIP RULE ENGINE

VIII.A. Why Not Pre-Compute Golden Records

Most MDM systems compute and store a static "golden record" — a single merged view of the best attributes from all contributing source records. The problem with this approach: if you change a survivorship rule (say, switching from "most recent wins" to "source system X always wins" for the address field), you have to recompute the golden record for every cluster that has a contributing address from source X. For millions of records, that's a batch job.

In Averio MDM, golden records are not stored. They are assembled from source records at query time by applying the configured survivorship rules. Rule changes take effect immediately for all subsequent retrievals — no batch recompute.

VIII.B. The Six Rule Types

`SurvivorshipEngine.buildGoldenRecord()` applies one of six strategies per attribute:

19. SOURCE_PRIORITY: The attribute value from the highest-ranked available source system wins.
20. MOST_RECENT: The attribute value with the most recent `sourceLastUpdated` timestamp wins.
21. MOST_FREQUENT: Majority vote across source records — the value that appears most often wins.
22. SUPREMACY: A designated source system always wins, regardless of recency or frequency.
23. NON_NULL: The first non-null value from sources in priority order wins.

24. LONGEST: The longest string value wins — useful for address fields where more detail is generally better.

VIII.C. View-Scoped Golden Records

When a `viewId` is supplied, `GoldenRecordService.getGoldenRecordForView()` loads survivorship rules filtered to that view. Different business consumers — risk, compliance, marketing, operations — can have different rule configurations applied to the same underlying source data. A risk team might want the most conservative (most recent) address. A marketing team might want the longest address. Both get their own golden view from the same source records.

IX. HUMAN-IN-THE-LOOP ML RETRAINING

IX.A. Feature Capture at Decision Time

This is a subtle but important point. When a steward reviews a record pair and decides whether they match, that decision is based on the attributes as they exist at that moment. If you wait until later to extract features for training — say, you re-query the database at training time — the records may have changed. You'd be training on features that don't match what the steward actually saw.

`MLMatchingService.captureDecision()` extracts and persists the feature vector at the exact moment the steward submits their decision:

```
nameSimilarity, dobExactMatch, taxIdExactMatch, emailMatch, phoneMatch,
addressSimilarity, dunsMatch, leiMatch, nationalIdMatch,
sourceSystemDiversity, partyTypeMatch
```

These are stored as an immutable `MatchingFeedback` document with the steward's label (MATCH or NO_MATCH). They never change after that point, regardless of what happens to the party records later.

This design enables **replay-free retraining** — the training set is fully self-contained in the feedback documents. You don't need the original party states to retrain.

IX.B. Automatic Retraining

After each feedback save:

```
if (feedbackCount >= MIN_EXAMPLES && feedbackCount % RETRAIN_EVERY == 0) {
    retrainModel(entityType);
}
```

With `MIN\_EXAMPLES = 5` and `RETRAIN\_EVERY = 5`, the model retrains every 5 labeled examples once the minimum threshold is met.

IX.C. Logistic Regression Implementation

The retrain uses a straightforward logistic regression with gradient descent:

- Feature matrix $X \in \mathbb{R}^{(n \times d)}$, $d = 11$ features
- Up to 500 gradient descent iterations
- Learning rate $\eta = 0.1$
- L2 regularization $\lambda = 0.01$ on feature weights (not bias)
- Convergence test on loss change

After training: accuracy, precision, recall, F1, and per-feature importance ($|w_i| / \sum |w_j|$) are computed and stored with the model.

X. NAME NORMALIZATION

X.A. Organization Name Normalization

`NameNormalizerService.normaliseOrg()` strips legal suffixes before blocking key generation: `"IBM Corporation, Inc."` $\rightarrow$ `"ibm"`. It handles 30+ suffixes in multiple languages using longest-match-first to avoid partial matches (preventing "Corp" from matching before "Corporation" in the same string).

Without this, "IBM Corp" and "IBM Corporation" would produce different blocking keys and never be compared. With it, they both reduce to "ibm" and share all phonetic blocking keys.

X.B. Nickname Equivalence

`NicknameService` maintains 90+ equivalence groups for common English given names with international variants. "Bob", "Robert", "Rob", "Bobby", "Robby", and "Roberto" are all members of the same group. Methods:

- `variants(firstName)` — returns all equivalent forms
 - `areEquivalent(a, b)` — boolean equivalence check
 - `similarity(a, b)` — returns 1.0 for equivalents, 0.0 otherwise
-

XI. IN-APPLICATION TEST LABORATORY

The platform includes a test framework (`com.averio.mdm.testing`) that runs against the live application infrastructure. This matters because tests against mocked databases frequently pass while production bugs persist — the mock is not the database.

Seven suites:

25. `API_HEALTH` — connectivity checks across all services and data stores

26. `MATCHING` — matching algorithm correctness tests (in-memory, no DB I/O)

- 27. BLOCKING — blocking key generation and index operation tests (in-memory)
- 28. SURVIVORSHIP — rule application correctness tests
- 29. GOLDEN_RECORD — golden record service tests with actual Cosmos DB writes
- 30. TIMELINE — event persistence and ordering tests with per-test cleanup
- 31. REGRESSION — end-to-end ingest scenarios with Neo4j + Cosmos DB and automatic test data cleanup

Test records are isolated via ``sourceSystem = "TEST_LAB"``` and ``sourceSystemId = "TEST-{runId}-{seq}"``` markers. Dependencies use ``@Autowired(required=false)``` with null-guard ``SKIPPED``` status for unavailable services.

CLAIMS

The following claims are submitted for the provisional record. These will be formalized for the non-provisional application.

Claim 1. A computer-implemented method for entity resolution comprising:

- generating, from a party record, blocking keys using nine independent strategies applied in union, the strategies including phonetic name encoding, nickname variant phonetic encoding, date-of-birth hashing, tax identifier prefix, phone suffix, email domain plus initial, postal code plus phonetic, collapsed name phonetic, and exact high-cardinality identifier matching;
- retrieving candidate records sharing at least one blocking key;
- scoring candidates using Fellegi-Sunter parameters that are autonomously estimated by an Expectation-Maximization algorithm without labeled training data;
- routing the top match to auto-link, steward review, or new entity creation based on configurable score thresholds; and
- when routing to steward review, assigning a provisional globally unique golden identifier to the incoming record before committing any database transaction.

Claim 2. A computer-implemented method for cluster drift detection comprising:

- upon update of a party record, scoring the updated record against all sibling records in its current cluster;
- when the best intra-cluster score falls below a threshold, determining drift and executing a full cross-cluster candidate search;
- automatically reassigning, escalating, or detaching the record based on the external search results; and
- recording all membership changes as immutable timeline events indexed by golden record identifier.

Claim 3. A system for autonomous Fellegi-Sunter parameter estimation comprising:

- an Expectation-Maximization algorithm that estimates m-probabilities and u-probabilities from unlabeled record pairs;
- a scheduled trigger for recurring re-estimation;
- a sanity validation guard that reverts to prior values on aberrant results; and
- a per-party-type parameter store with estimation timestamps.

Claim 4. A method for human-in-the-loop ML retraining comprising:

- capturing feature vectors at the time of human review decisions, not at training time;
- persisting feature vectors with human labels as immutable records;
- automatically triggering logistic regression retraining upon accumulation of new labeled examples; and
- computing and storing model performance metrics including precision, recall, and per-feature importance.

Claim 5. A method for provisional golden identity assignment comprising:

- assigning a globally unique golden identifier to every ingested record before any database transaction commits;
- creating a complete golden record for provisional records;
- making provisional records fully available to downstream systems during review; and
- upon merge approval, migrating to the surviving cluster identifier; upon merge rejection, converting the provisional identifier to permanent without additional operations.

Claim 6. The method of Claim 1, wherein scoring includes a third AI-enhanced stage using a Large Language Model invoked conditionally only for scores in an ambiguous range, with the final score as a weighted blend of probabilistic and LLM scores.

Claim 7. The method of Claim 1, wherein nickname variant encoding uses a stored table of equivalence groups covering 90+ given name families, generating phonetic blocking keys for each equivalent form.

Claim 8. The method of Claim 1, wherein the blocking index is maintained as dual concurrent hash maps — an inverted index mapping keys to record sets and a forward index mapping records to key sets — enabling $O(1)$ complete removal of a record from the blocking index.

Claim 9. The method of Claim 2, wherein timeline events are persisted as structured documents to a distributed document store partitioned by golden record identifier, containing event type, prior state, new state, timestamp, actor, and event-specific metadata.

Claim 10. The system of Claim 3, wherein the E-step uses log-space arithmetic with a maximum-log offset to prevent floating-point underflow during posterior probability computation.

Claim 11. The system of Claim 3, further comprising a query-time survivorship rule engine that assembles golden records at retrieval time by applying configurable per-attribute rules from: source priority ordering, most-recent timestamp, majority vote, supremacy source, first non-null, or longest string; enabling rule changes to take effect immediately without data layer recomputation.

Claim 12. The method of Claim 4, wherein feature vectors captured at decision time preserve the statistical basis for the human label independent of subsequent mutations to the underlying records, enabling replay-free retraining.

Claim 13. The method of Claim 5, wherein the provisional golden identifier is a system-generated zero-padded numeric string, and wherein a client-supplied identifier in any format may override the auto-generated identifier during migration from legacy systems.

Claim 14. A method for transitive entity cluster formation using Union-Find with path compression and union-by-rank, wherein any two records transitively connected through a chain of pairwise matches are assigned to the same cluster regardless of whether they were directly compared.

Claim 15. The method of Claim 1, wherein probabilistic scoring applies twenty or more similarity algorithms including Jaro-Winkler, token sort ratio, token set ratio, bigram Jaccard, trigram Jaccard, Damerau-Levenshtein, Double Metaphone, TF-IDF character n-gram cosine similarity, and nickname equivalence lookup, with outputs combined using Fellegi-Sunter log-likelihood weighting.

ABSTRACT

Averio MDM is an enterprise Master Data Management platform built around five technical innovations for entity resolution at scale. A nine-strategy union blocking engine brings more true match candidates into scope than single-strategy approaches by applying phonetic, demographic, identifier, geographic, and contact strategies simultaneously and taking their union. A nightly Expectation-Maximization algorithm re-calibrates the Fellegi-Sunter match model parameters from unlabeled data, keeping accuracy stable as data characteristics evolve over time. A three-stage cascading pipeline routes records through deterministic, probabilistic, and AI-enhanced matching in sequence, using Large Language Model disambiguation only for the statistically ambiguous subset. A provisional golden identity system assigns every record a permanent identifier before its first database commit, so no record is ever in a null identity state regardless of where it sits in the review process. A post-update drift detection algorithm re-evaluates cluster membership on every attribute update and automatically corrects incorrect merges as data quality improves, recording all membership changes as an immutable timeline for compliance audit.

INVENTOR DECLARATION

I, Suvojeet Pal, declare that I am the sole inventor of the subject matter described and claimed in this provisional patent application. I have conceived and developed the Averio MDM system described herein as an individual. The system reflects my own original technical design based on direct experience with the limitations of existing enterprise Master Data Management products.

I acknowledge the duty to disclose information material to patentability as defined in 37 C.F.R. § 1.56.

Inventor:

Name: Suvojeet Pal

Residence: [Address]

Citizenship: [Citizenship]

Signature: _____ Date: _____

PRIOR ART CONSIDERED

U.S. Patents

- US 7,853,573 B2 — Data integration system (IBM InfoSphere)
- US 8,285,719 B2 — Entity resolution in master data management
- US 9,330,169 B2 — Probabilistic record linkage with feedback learning
- US 10,437,882 B2 — Dynamic survivorship rules for master data

Published Literature

32. Fellegi, I.P. and Sunter, A.B. (1969). A theory for record linkage. *Journal of the American Statistical Association*, 64(328), 1183-1210.
33. Winkler, W.E. (1988). Using the EM algorithm for weight computation in the Fellegi-Sunter model. *Proceedings ASA Survey Research Methods*, 667-671.
34. Larsen, M.D. and Rubin, D.B. (2001). Iterative automated record linkage using mixture models. *JASA*, 96(453), 32-41.
35. Damerau, F.J. (1964). A technique for computer detection and correction of spelling errors. *CACM*, 7(3), 171-176.
36. Christen, P. (2012). *Data Matching*. Springer.
37. Köpcke, H. and Rahm, E. (2010). Frameworks for entity matching. *Data & Knowledge Engineering*, 69(2), 197-210.

How This Invention Differs

Over US 9,330,169: That system requires labeled training pairs. This invention learns parameters unsupervised via EM on a schedule.

Over all blocking prior art: No prior reference discloses nine independent strategies in union with a dual inverted/forward concurrent hash map index. Prior systems use two to four strategies at most.

Over US 10,437,882: That system pre-computes and stores golden records. This invention assembles them at query time, enabling instant rule changes and view-scoped golden records.

Over all commercial MDM systems: No prior art discloses the Provisional Golden Identity pattern. All known commercial MDM platforms (IBM InfoSphere MDM, Informatica MDM, SAP Master Data Governance, TIBCO EBX, Reltio Cloud) allow records in steward review to exist without a golden identifier.

Over all commercial MDM systems: No prior art discloses post-update probabilistic cluster drift detection. Existing systems leave incorrect merges in place indefinitely after the attributes that caused the merge are corrected.

Over LLM-based matching proposals: No prior art discloses conditional three-stage cascading that uses LLM disambiguation only for the ambiguous score range, preserving deterministic and probabilistic performance for the majority of cases while using AI as a targeted fallback.